INTERNSHIP PROJECT REPORT

Big Data Analytics Basics Using PySpark

Internship: Data Analytics Internship

Organization: Edutech Solution

Task Focus: Task 15 – Big Data Analytics Basics

Prepared By: Saloni Kaushal

Tools Utilized: Google Colab, Apache Spark (PySpark), Python

Dataset Profile: client_hostname.csv

Date of Submission: July 2, 2026

Acknowledgement

I wish to express my deep sense of gratitude and respect to **Edutech Solution** for providing me with the exceptional opportunity to undertake this project as a part of my Data Analytics Internship. This project, titled **"Big Data Analytics Basics Using PySpark,"** has been an invaluable learning curve that allowed me to transition theoretical concepts of big data engineering into practical, industrial-grade implementation.

I extend my heartfelt thanks to my industrial mentors and technical supervisors at Edutech Solution for their continuous guidance, insightful feedback, and structured curriculum design. Their guidance during technical blockers—particularly around distributed computing paradigms, cluster environment configuration within Google Colab, and Spark dataframe optimizations—was crucial to completing this milestone.

I am also incredibly grateful to my academic advisors and peers who provided the intellectual infrastructure necessary to conduct this study. This work serves not only as a reflection of my technical growth during this internship but also as an analytical framework for enterprise network log data processing.

— Saloni Kaushal

Table of Contents

4. Executive Summary

This enterprise-grade technical report covers the architectural paradigms, implementation, and analytical findings of the project **"Big Data Analytics Basics Using PySpark,"** completed during my Data Analytics Internship at **Edutech Solution**.

The main objective of this study is to implement distributed computing techniques using Apache Spark's Python API (PySpark) to ingest, profile, cleanse, transform, and analyze network lookup log datasets. The target file, `client_hostname.csv`, contains a high-volume profile of **258,445 network transaction records** documenting client IP addresses, resolved hostnames, routing aliases, and nested destination addresses.

EXECUTIVE SUMMARY METRICS	
Total Ingested Records	258,445 Rows
Data Schema Columns	4 Core Features
Target Processing Engine	Apache Spark Core (v4.0.3) via PySpark
Runtime Environment	Ephemeral Distributed Virtual Machine
Primary Operations	Missing Value Audits, High-Cardinality GroupBy Aggregations, Ordered Sorting

The data was processed using an integrated cloud stack comprising Python 3 and Apache Spark within a Google Colab virtual machine instance. Throughout the engine's execution lifecycle, structural audits were conducted to uncover major quality issues: **97.74% of the routing infrastructure entries exhibited resolution failures** (flagged via `[Errno 1] Unknown host`), and a matching **97.74% null rate** was observed within the destination physical IP mapping matrices. By applying distributed multi-node operations such as grouping, counting, and transformations, this project showcases how PySpark handles high-cardinality lookups, isolates broken network endpoints, and scales up data processing far beyond the limits of traditional single-node tools.

5. Introduction

5.1 What is Big Data?

Big Data refers to datasets whose volume, velocity, variety, and structural complexity transcend the storage capacity, processing capability, and analytical threshold of traditional relational database management systems (RDBMS) or standalone hardware infrastructures. It represents an explosive proliferation of data generated by modern interconnected systems, including enterprise server logs, telecommunication transactions, internet-of-things (IoT) sensor streams, and social media repositories.

5.2 Why Big Data is Important

In contemporary business ecosystems, data acts as a foundational asset for strategic engineering and predictive modeling. Big Data allows enterprises to uncover hidden correlations, detect systemic anomalies, optimize supply chains, and build automated machine learning systems. Processing this data helps organizations transform raw, messy logs into actionable metrics, lowering operational risk and improving system reliability.

5.3 Applications of Big Data

- **Network Infrastructure & Cybersecurity:** Real-time log ingestion to map global host addresses, trace request origins, and stop distributed denial-of-service (DDoS) vectors.
- **Financial Risk Engineering:** Multi-node fraud detection models processing millions of transactional events per second.
- **Healthcare Genomics:** Distributed alignment of complex sequences containing billions of data points.

5.4 Need for Distributed Computing

Traditional computation runs on a symmetric multiprocessing architectural design, which scales vertically (Scale-Up) by adding faster CPUs and memory to a single machine. However, vertical scaling faces physical engineering bottlenecks, high costs, and single-point-of-failure vulnerabilities.

Distributed computing addresses these limitations through horizontal scaling (Scale-Out). It links groups of standard server nodes via high-speed local networks to share data storage and computation tasks. By breaking large datasets into smaller chunks across an unshared architecture, distributed computing processes data in parallel, ensures fault tolerance, and easily scales to petabyte workloads.

6. Objectives

The strategic objectives of this project are:

1. **Environment Setup:** Build an optimized Apache Spark application within an ephemeral cloud runtime using Google Colab.
2. **Distributed Data Ingestion:** Load the `client_hostname.csv` log file into an immutable Spark DataFrame using lazy evaluation and automated schema inference.
3. **Data Quality Profile:** Calculate exact completeness indices by identifying actual `null` pointer exceptions and explicit `"NULL"` text entries.
4. **Statistical Inversion:** Evaluate categorical counts and frequency distributions for network entities like `client` IPs and `hostname` properties.
5. **Compute Optimization:** Design high-performance aggregation and sorting queries that take advantage of Spark's catalyst optimizer.

6. **Industrial Documentation:** Author a professional engineering report suitable for academic review and corporate deployment.

7. Big Data Concepts

7.1 Traditional Data vs. Big Data

PARAMETRIC INDEX	TRADITIONAL DATA	BIG DATA
Scale Volume	Megabytes to Gigabytes	Terabytes to Petabytes
Architecture	Centralized/RDBMS	Distributed/De-centralized
Structural Model	Highly Structured (Tables)	Unstructured/Semi-Structured
Tooling Pipeline	SQL Server, Oracle, MySQL	Spark, Hadoop, NoSQL, Hive
Scalability Mode	Vertical (Scale-Up Memory)	Horizontal (Scale-Out Nodes)

7.2 Characteristics of Big Data (The 5 Vs)

- **Volume:** Measures the sheer scale of generated data. It requires distributed storage systems like HDFS or cloud object fabrics because files easily surpass the terabyte mark.
- **Velocity:** Defines the speed at which data is created, ingested, and processed. It spans from batch processing intervals to micro-batching and real-time streaming pipelines.
- **Variety:** Covers the various formats of modern data, which can be highly structured SQL files, semi-structured JSON or XML files, or completely unstructured data like network traffic logs and binary text streams.
- **Veracity:** Addresses data cleanliness and trustworthiness. It requires sophisticated data engineering practices to handle noise, missing fields, malformed lines, and anomalous entries.
- **Value:** Focuses on the business impact of data processing. It ensures that the costs of distributed computing pay off by uncovering insights that optimize system performance and reduce risks.

8. Hadoop Overview

8.1 Hadoop Distributed Architecture

Apache Hadoop is an open-source framework designed for the distributed storage and processing of massive datasets across clusters of commodity hardware. It shifts data processing away from high-end proprietary servers to standard computing nodes, relying on software configurations to provide fault tolerance and data reliability.

8.2 Core Ecosystem Components

MAPREDUCE: Distributed Software Processing Layer via Map/Reduce workflow.

YARN: Yet Another Resource Negotiator - Cluster Resource Allocator.

HDFS: Hadoop Distributed File System - Fault-Tolerant Storage Fabric.

- **HDFS:** A distributed file system that splits large data files into fixed-size blocks (typically 128MB) and replicates them across multiple cluster nodes to protect against hardware failures.
- **YARN:** The resource management layer that schedules jobs and allocates system resources (CPU cores and memory) across the cluster.
- **MapReduce:** A software framework and software model used to process large amounts of data in parallel using a two-phase map and reduce workflow.

8.3 Advantages and Limitations of Hadoop

Advantages: Exceptionally cost-effective for long-term storage of massive datasets, handles arbitrary data formats, and scales out horizontally to thousands of nodes.

Limitations: Relying on hard drives for intermediate MapReduce steps causes significant disk I/O bottlenecks. It lacks interactive query performance, does not support real-time streaming, and requires complex, low-level Java code to maintain.

9. Apache Spark and PySpark

9.1 Introduction to Apache Spark

Apache Spark is a lightning-fast, multi-language data processing engine built for large-scale data engineering, data science, and machine learning. Its major innovation is stateful **in-memory data processing**, which allows applications to cache data in RAM across the cluster. This bypasses the disk read/write bottlenecks common in traditional systems like Hadoop MapReduce and accelerates iterative workloads by up to 100 times.

9.2 Spark Component Ecosystem

- **Spark Core:** The central foundation of the engine, managing memory allocation, task scheduling, fault recovery, and interactions with storage systems.
- **Spark SQL & DataFrames:** A module that provides structured data abstractions and an optimized execution planner called Catalyst.
- **Spark Streaming:** Enables scalable, high-throughput, fault-tolerant processing of real-time live data streams.
- **MLlib:** A scalable machine learning library providing distributed learning algorithms.
- **GraphX:** A unified API for distributed graph computation and structural analysis.

9.3 PySpark Architecture & API Foundations

PySpark is the official Python API for Apache Spark. It allows data engineers to combine the simplicity and rich ecosystem of Python with the distributed computing power of Spark. PySpark utilizes the Py4J library to enable seamless communication between Python scripts running on the client machine and the JVM engine operating across the Spark cluster.

9.4 Advantages of Spark over Hadoop

1. **Processing Velocity:** In-memory execution avoids writing intermediate states to physical disks.
2. **Lazy Evaluation:** Computation tasks are tracked using a **Directed Acyclic Graph (DAG)**. Spark optimizes this plan before running any jobs, minimizing data movement.
3. **Rich API Ecosystem:** Out-of-the-box support for SQL queries, streaming data, and machine learning, replacing the complex Java code required by MapReduce.

9.5 Core Mechanics of Spark DataFrames

A Spark DataFrame is a distributed collection of data organized into named columns, conceptually equivalent to a table in a relational database but backed by distributed processing. DataFrames rely on **Resilient**

Distributed Datasets (RDDs) as their underlying foundation. RDDs are fault-tolerant, immutable collections of objects partitioned across cluster nodes, allowing Spark to transparently reconstruct data if a worker node fails.

10. MapReduce Concept

10.1 Mathematical Workflow

The MapReduce model processes data through three main functional transformations:

$$\textit{Map Component: } (K_1, V_1) \rightarrow \textit{List}(K_2, V_2)$$

$$\textit{Shuffle Component: } \textit{List}(K_2, V_2) \rightarrow \textit{List}(K_2, \textit{List}(V_2))$$

$$\textit{Reduce Component: } (K_2, \textit{List}(V_2)) \rightarrow \textit{List}(K_3, V_3)$$

- **Map Phase:** Reads data fragments as key-value pairs, filters or transforms them, and emits intermediate key-value entries.
- **Shuffle Phase:** Aggregates, sorts, and routes all intermediate values sharing an identical key across network nodes to the same reducer.
- **Reduce Phase:** Aggregates the shuffled collections to compute final metrics, which are then saved back to persistent storage.

10.2 Word-Count Execution Trace

```
Input Stream:  ["Spark Client", "Client Session"]

1. Map Execution:
   - Line 1 -> ("Spark", 1), ("Client", 1)
   - Line 2 -> ("Client", 1), ("Session", 1)

2. Shuffle & Aggregation:
   - "Client"  -> [1, 1]
   - "Session" -> [1]
   - "Spark"   -> [1]

3. Reduce Execution:
   - ("Client", 2)
   - ("Session", 1)
   - ("Spark", 1)
```

11. Dataset Description

11.1 Structural Profile & Metrics

The reference file `client_hostname.csv` documents internet network endpoint operations and lookup outcomes. The structural profile of this dataset includes:

FILE STRUCTURE INDEX	METRIC DETAILS
Targeted Resource Path	/content/client_hostname.csv
Quantitative Row Count	258,445 Records
Dimensional Column Count	4 Attributes
Structural Ingestion Mode	CSV with Header, Inferred Schema

11.2 Schema and Data Types

The data frame structures and attributes inferred by the PySpark engine are detailed below:

```
root
 |-- client: string (nullable = true)
 |-- hostname: string (nullable = true)
 |-- alias_list: string (nullable = true)
 |-- address_list: string (nullable = true)
```

- **`client` (String):** Represents the requesting source internet protocol (IP) address initiating the query.
- **`hostname` (String):** The target server name or resolved alphanumeric domain endpoint.
- **`alias_list` (String):** Alternative domain routing aliases; frequently records error messages like `[Errno 1] Unknown host` when resolutions fail.
- **`address_list` (String):** A string representation of a list containing the destination physical IP addresses mapped to the hostname.

[Insert Figure 1: Dataset Profile and Schema Constraints Screenshot Here]

12. Methodology

The methodology uses an end-to-end distributed data engineering approach. It starts with setting up the virtual machine environment, ingests the data into Spark, and performs non-destructive analysis and query sorting.

[Insert Figure 2: End-to-End Computational Workflow Architectural Diagram]

13. Implementation

This section details the code blocks used in the project, explaining their technical rationale and objectives.

Step 1: Install PySpark Environment

Objective: Prepare the runtime environment by downloading and installing Apache Spark modules.

```
!pip install pyspark
```

Rationale: Installs PySpark version 4.0.3 and the Py4J dependency into Google Colab's default environment, enabling distributed computation.

Step 2: Import API Classes

Objective: Expose the necessary object classes to manage the underlying cluster.

```
from pyspark.sql import SparkSession
```

Step 3: Instantiate SparkSession Engine

Objective: Build and configure a local coordinate context framework.

```
spark =
SparkSession.builder     .appName("BigDataAnalyticsBasics")     .getOrCreate()

print("SparkSession created successfully!")
```

Step 4: Confirm Dataset Ingestion Paths

Objective: Verify that the target CSV file is present in the cloud runtime directory.

Step 5: Read CSV into Spark DataFrame

Objective: Transform raw text records into an immutable, partitioned collection.

```
dataset_path = "/content/client_hostname.csv"
df = spark.read.csv(dataset_path, header=True, inferSchema=True)
print(f"Dataset '{dataset_path}' loaded into Spark DataFrame.")
```

Step 6: Visual Inspection of Records

Objective: Display a sample of the data to verify column alignment and contents.

```
df.show(10)
```

Step 7: Struct Schema Reflection

Objective: Output the structural data types of each column.

```
df.printSchema()
```

Step 8: Total Count Quantifications

Objective: Measure the overall size of the dataset.

```
total_rows = df.count()
print(f"Total number of rows in the DataFrame: {total_rows}")
```

Step 9: Column Dimension Calculations

Objective: Calculate the total number of features in the schema.

```
total_columns = len(df.columns)
```

Step 10: Column Header Inspection

Objective: Output an ordered list of all column labels.

```
column_names = df.columns
for col_name in column_names:
    print(f"- {col_name}")
```

Step 11: Data Completeness Audit

Objective: Identify and count missing data values across all features.

```
from pyspark.sql.functions import col
missing_values_counts = []
for column in df.columns:
    null_count = df.filter(col(column).isNull()).count()
    if dict(df.dtypes)[column] == 'string':
        null_string_count = df.filter(col(column) == 'NULL').count()
    else:
        null_string_count = 0
    total_missing = null_count + null_string_count
    if total_missing > 0:
        missing_values_counts.append((column, total_missing))
```

Step 12: Duplicate Entry Analysis

Objective: Check for exact duplicate rows in the dataset.

```
distinct_rows = df.distinct().count()
duplicate_rows = df.count() - distinct_rows
```

Step 13: Summary Statistics Production

Objective: Generate a high-level statistical overview of the dataset.

```
df.describe().show()
```

Step 14: Exploratory Data Analysis (EDA)

Objective: Uncover distribution patterns and find the most frequent entries across columns.

```
from pyspark.sql.functions import countDistinct, desc
df.groupBy("hostname").count().orderBy(desc("count")).show(10)
```

Step 15: Structural GroupBy Operations

Objective: Group data by unique combinations of columns.

```
df.groupBy("client",
"hostname").agg(count("client").alias("combination_count"))   .orderBy(desc("combinatio
```

Step 16: Column Sort Optimizations

Objective: Order the rows alphabetically by specific columns.

```
from pyspark.sql.functions import asc
sorted_df = df.orderBy(asc("client"), asc("hostname"))
```

14. Results and Analysis

This section analyzes the outputs generated by running the PySpark commands.

14.1 Dataset Structure and Ingestion Validation

The ingestion step ran without errors, outputting the message: `"Dataset '/content/client_hostname.csv' loaded into Spark DataFrame."` The file structure contains **258,445 rows** and **4 columns**.

14.2 Initial Row Samples

Printing the first 10 rows reveals key data patterns and quality issues:

```
+-------------+--------------------+--------------------+-----------------+
|       client|            hostname|          alias_list|     address_list|
+-------------+--------------------+--------------------+-----------------+
| 5.123.144.95|        5.123.144.95|[Errno 1] Unknown...|             NULL|
| 5.122.76.187|        5.122.76.187|[Errno 1] Unknown...|             NULL|
| 5.215.249.99|        5.215.249.99|[Errno 1] Unknown...|             NULL|
|31.56.102.211|31-56-102-211.sha...|['211.102.56.31.i...|['31.56.102.211']|
|5.123.166.223|       5.123.166.223|[Errno 1] Unknown...|             NULL|
|  5.160.26.98|         5.160.26.98|[Errno 1] Unknown...|             NULL|
|5.127.147.132|       5.127.147.132|[Errno 1] Unknown...|             NULL|
|158.58.30.218|       158.58.30.218|[Errno 1] Unknown...|             NULL|
| 86.55.230.86|        86.55.230.86|[Errno 1] Unknown...|             NULL|
| 89.35.65.186|        89.35.65.186|[Errno 1] Unknown...|             NULL|
+-------------+--------------------+--------------------+-----------------+
```

14.3 Completeness and Missing Value Audit

The missing value check revealed severe data gaps in the network log file:

- **`hostname` column:** 4 missing values.
- **`address_list` column:** 252,626 missing values.

Out of 258,445 total rows, only 5,819 contain valid data in the `address_list` column, meaning **97.74% of the fields are missing or null**.

[Insert Figure 3: Missing Value Frequency and Structural Gaps Bar Chart]

14.4 Summary Statistics Matrix

Running the descriptive statistics command generated the following output matrix:

```
+-------+------------+--------------------+--------------------+------------------+
|summary|      client|            hostname|          alias_list|      address_list|
+-------+------------+--------------------+--------------------+------------------+
|  count|      258445|              258441|              258445|              5819|
|   mean|        NULL|                NULL|                NULL|              NULL|
| stddev|        NULL|                NULL|                NULL|              NULL|
|    min|1.132.107.223|014199038003.ctin...|['0.167.77.40.in-...| ['1.159.185.202']|
|    max|99.99.188.195| zx2.quadmetrics.com|[Errno 1] Unknown...|['99.253.184.236']|
+-------+------------+--------------------+--------------------+------------------+
```

Because all columns contain string data, numeric averages and standard deviations are absent (`NULL`). The minimum and maximum fields show the alphabetical boundaries of the text data.

14.5 Exploratory Data Inversion (EDA)

Client Column Profile

The dataset contains **258,445 unique client IP addresses**. Every client row contains exactly one record, meaning individual source IPs do not repeat across logs.

Hostname Column Profile

The dataset contains **258,273 unique hostnames**. Unlike clients, certain network endpoints appear multiple times:

```
+---------------------------------+-----+
|hostname                         |count|
+---------------------------------+-----+
|int0.client.access.fanaptelecom.net|101  |
|unknown.puregig.net              |31   |
|server2us.getmyip.co             |5    |
|swe-net-ip.as51430.net           |5    |
|NULL                             |4    |
|zmap.sorengard.com               |4    |
+---------------------------------+-----+
```

The endpoint `int0.client.access.fanaptelecom.net` is the most active host in the log, appearing **101 times**.

Alias List Error Frequency

The audit found that the error message `[Errno 1] Unknown host` occurs **252,626 times** within the `alias_list` column. This confirms that **97.74% of the log entries represent failed domain name resolutions** rather than valid alternative routing names.

[Insert Figure 4: Top 10 Most Frequent Hostnames and Domain Distribution Screenshot Here]

14.6 Sorting Verification

Sorting the dataset in ascending order aligns the rows alphabetically by IP address, grouping similar network subnets together for easier analysis:

```
+-------------+----------------------------------------+------------------------------+-------------
|client       |hostname                                |alias_list                    |address_list
+-------------+----------------------------------------+------------------------------+-------------
|1.132.107.223|1.132.107.223                           |[Errno 1] Unknown host        |NULL
|1.132.108.133|1.132.108.133                           |[Errno 1] Unknown host        |NULL
|1.136.111.52 |1.136.111.52                            |[Errno 1] Unknown host        |NULL
+-------------+----------------------------------------+------------------------------+-------------
```

15. Key Findings

1. **High Volume Network Footprint:** The dataset contains a substantial log history of **258,445 transaction rows**, requiring distributed processing engines like PySpark to avoid single-node performance bottlenecks.
2. **Extreme Client Dispersion:** There are **258,445 unique client IPs** in the log, indicating that each record represents a request from a distinct network endpoint.
3. **Severe Resolution Failure Rates:** A total of **252,626 entries (97.74%)** contain the `[Errno 1] Unknown host` error message, indicating widespread connection or DNS lookup issues across the recorded infrastructure.
4. **Widespread Data Gaps:** The `address_list` column has a **97.74% null rate**, meaning only 5,819 records successfully resolved and mapped to a destination physical IP address.
5. **Clear Network Hotspots:** The domain name `int0.client.access.fanaptelecom.net` appears **101 times**, making it the most frequent endpoint and a primary target for connectivity optimization.
6. **Presence of Malformed Identifiers:** The string `"NULL"` appears **4 times** inside the `hostname` column instead of a valid domain name, highlighting the need for stricter data validation.
7. **Clean Deduplication Profile:** A complete distinct count audit showed **zero duplicate records**, confirming that each row represents a unique network request.
8. **Consistent Column Structures:** All four attributes in the schema (`client`, `hostname`, `alias_list`, `address_list`) contain string data, meaning text-based functions are required for cleaning and transformation tasks.
9. **Traceable Domain Structures:** Successful lookups often show structured domain properties (such as `.net` or `.com` suffixes), while failed lookups frequently show the client's raw IP address copied directly into the hostname column.
10. **Predictable Subnet Ranges:** Sorting the dataset alphabetically clusters related client IPs together, making it easier for network teams to isolate bad traffic or faulty subnets.

16. Challenges Faced

- **PySpark Environment Provisioning:** Configuring Java development kits and Spark environment path variables inside Google Colab's ephemeral virtual machine requires careful setup to avoid package mismatches during installation.
- **Literal vs. Structural Null Handling:** Distinguishing between actual database null pointers (`None`/`null`) and literal text strings (`"NULL"`) requires custom filter rules to calculate accurate data completeness metrics.

- **Distributed Processing Logic:** Transitioning from traditional single-node tools (like Pandas) to PySpark requires a shift in mindset to account for lazy evaluation, execution plans (DAGs), and minimizing data shuffle across networks.
- **Memory Management During Sorting:** Sorting high-cardinality string variables like IP addresses can trigger heavy data shuffling across virtual partitions, requiring careful control over cluster memory to prevent out-of-memory errors.

17. Learning Outcomes

- **Distributed Framework Design:** Gained a deep understanding of Apache Spark's core architecture, including how driver nodes assign tasks to distributed worker executors.
- **Optimized Programming Techniques:** Mastered the use of PySpark DataFrames, lazy evaluation principles, and how the Catalyst Optimizer evaluates execution plans before running jobs.
- **Advanced Data Inversion:** Developed practical skills in data profiling and quality auditing by isolating wide-scale resolution errors and structural data gaps.
- **Enterprise Cloud Deployment:** Learned to spin up, configure, and maintain virtual processing clusters inside Google Colab environments.
- **Industrial Problem Solving:** Acquired the ability to analyze complex corporate log files and convert raw text data into clear operational insights.

18. Future Scope

- **Real-Time Stream Processing:** Upgrade the batch processing architecture to use **Spark Structured Streaming** to ingest and evaluate network lookup logs in real time.
- **Cloud Infrastructure Migration:** Move the local PySpark script to enterprise cloud platforms like AWS EMR, Google Cloud Dataproc, or Azure HDInsight to test scalability across multi-node clusters.
- **Automated Machine Learning Integration:** Connect the data pipeline to **Spark MLlib** to train predictive security models that can flag abnormal request patterns or detect DDoS attacks as they happen.
- **Advanced Extraction Pipelines:** Use Spark regular expressions and user-defined functions (UDFs) to parse nested string fields like `alias_list` and extract clean, actionable sub-domains.
- **Interactive Dashboard Connectivity:** Export the aggregated PySpark data outputs into business intelligence tools like Tableau or PowerBI to build live, interactive network performance dashboards.

19. Conclusion

This industrial project report successfully demonstrates the power and efficiency of **Apache Spark (PySpark)** for handling and analyzing large-scale log files. By loading and analyzing the **258,445 network transaction records** in the `client_hostname.csv` dataset, the application proved its value by processing complex data much faster than traditional single-node tools.

The data audit uncovered critical structural issues, showing a **97.74% lookup failure rate** across the network log history. It also pinpointed severe data gaps and identified specific domain hotspots. This internship project at **Edutech Solution** highlights how modern big data technologies can transform raw, messy network traffic logs into clear, actionable system insights, providing a strong foundation for future real-time analytics and predictive cloud monitoring systems.

20. References

1. Apache Spark Framework Architecture Documentation: https://spark.apache.org/docs/latest/
2. PySpark Developer API Reference Guide: https://spark.apache.org/docs/latest/api/python/
3. Apache Hadoop Distributed Storage Specification: https://hadoop.apache.org/docs/stable/
4. Google Colab Environment Runtime Configurations: https://colab.research.google.com/notebooks/welcome.ipynb
5. Python 3 Package Index and Documentation Registry: https://docs.python.org/3/